Online Shopping and Inventory System

DBMS Mini Project — MySQL 8.x

Overview

This system manages products, customers, orders, payments, stock and suppliers. It demonstrates joins, triggers, stored procedures and transactions on a normalized schema.

Tables

Table	Purpose	Key columns
Suppliers	Vendors who supply products	supplier_id (PK)
Customers	Registered shoppers	customer_id (PK), email (UNIQUE)
Products	Catalogue and stock levels	product_id (PK), supplier_id (FK), stock_qty
Orders	One row per customer order	order_id (PK), customer_id (FK), status
Order_Items	Products inside each order	item_id (PK), order_id (FK), product_id (FK)
Payments	Payments made against orders	payment_id (PK), order_id (FK)
Stock_Alerts	Low-stock log (trigger-filled)	alert_id (PK), product_id (FK)

Concepts Covered

• **Triggers:** stock check before insert, stock deduction and low-stock alert after insert, auto-mark order Paid.

• **Stored procedures:** place_order, add_item_to_order, cancel_order, make_payment, restock_product.

• **Transactions:** START TRANSACTION / COMMIT / ROLLBACK with an exit handler for all-or-nothing orders.

• **Joins:** INNER JOIN, LEFT JOIN and multi-table joins for reporting.

1. Table Definitions

```
DROP DATABASE IF EXISTS shop_db;
CREATE DATABASE shop_db;
USE shop_db;

CREATE TABLE Suppliers (
    supplier_id   INT AUTO_INCREMENT PRIMARY KEY,
    name          VARCHAR(100) NOT NULL,
    phone         VARCHAR(15),
    city          VARCHAR(50)
);

CREATE TABLE Customers (
    customer_id   INT AUTO_INCREMENT PRIMARY KEY,
    name          VARCHAR(100) NOT NULL,
    email         VARCHAR(100) UNIQUE NOT NULL,
    phone         VARCHAR(15),
    address       VARCHAR(200),
    created_at    DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE Products (
    product_id     INT AUTO_INCREMENT PRIMARY KEY,
    name           VARCHAR(100) NOT NULL,
    category       VARCHAR(50),
    price          DECIMAL(10,2) NOT NULL CHECK (price > 0),
    stock_qty      INT NOT NULL DEFAULT 0 CHECK (stock_qty >= 0),
    reorder_level  INT NOT NULL DEFAULT 5,
    supplier_id    INT,
    FOREIGN KEY (supplier_id) REFERENCES Suppliers(supplier_id)
);

CREATE TABLE Orders (
    order_id      INT AUTO_INCREMENT PRIMARY KEY,
    customer_id   INT NOT NULL,
    order_date    DATETIME DEFAULT CURRENT_TIMESTAMP,
    status        ENUM('Pending','Paid','Shipped','Delivered','Cancelled')
                  DEFAULT 'Pending',
    total_amount  DECIMAL(12,2) DEFAULT 0,
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
);

CREATE TABLE Order_Items (
    item_id      INT AUTO_INCREMENT PRIMARY KEY,
    order_id     INT NOT NULL,
    product_id   INT NOT NULL,
    quantity     INT NOT NULL CHECK (quantity > 0),
    unit_price   DECIMAL(10,2) NOT NULL,   -- price frozen at time of order
    FOREIGN KEY (order_id)   REFERENCES Orders(order_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);

CREATE TABLE Payments (
    payment_id    INT AUTO_INCREMENT PRIMARY KEY,
    order_id      INT NOT NULL,
    amount        DECIMAL(12,2) NOT NULL,
    method        ENUM('Card','UPI','NetBanking','COD') NOT NULL,
    payment_date  DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (order_id) REFERENCES Orders(order_id)
);

-- Filled automatically by a trigger when stock runs low
CREATE TABLE Stock_Alerts (
    alert_id     INT AUTO_INCREMENT PRIMARY KEY,
    product_id   INT,
    stock_left   INT,
    alert_time   DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);
```

2. Sample Data

```
INSERT INTO Suppliers (name, phone, city) VALUES
('TechSource Pvt Ltd', '9876500001', 'Delhi'),
('HomeGoods Traders',  '9876500002', 'Ludhiana'),
```

```
('BookWorld Distributors','9876500003','Mumbai');

INSERT INTO Customers (name, email, phone, address) VALUES
('Amit Sharma',  'amit@example.com',  '9811100001', 'Model Town, Ludhiana'),
('Priya Singh',  'priya@example.com', '9811100002', 'Sector 17, Chandigarh'),
('Rahul Verma',  'rahul@example.com', '9811100003', 'Civil Lines, Jalandhar'),
('Neha Gupta',   'neha@example.com',  '9811100004', 'Sarabha Nagar, Ludhiana');

INSERT INTO Products (name, category, price, stock_qty, reorder_level, supplier_id) VALUES
('Wireless Mouse',     'Electronics', 599.00,  50, 10, 1),
('USB-C Cable',        'Electronics', 249.00, 100, 20, 1),
('Bluetooth Speaker',  'Electronics',1999.00,  15,  5, 1),
('Steel Water Bottle', 'Home',        399.00,  40, 10, 2),
('Desk Lamp',          'Home',        899.00,   8,  5, 2),
('SQL Made Easy',      'Books',       499.00,  25,  5, 3);
```

3. Triggers

```
DELIMITER //

-- 3a. Block an order line if there is not enough stock
CREATE TRIGGER trg_check_stock
BEFORE INSERT ON Order_Items
FOR EACH ROW
BEGIN
    DECLARE available INT;
    SELECT stock_qty INTO available
    FROM Products WHERE product_id = NEW.product_id;

    IF available < NEW.quantity THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Insufficient stock for this product';
    END IF;
END//

-- 3b. Reduce stock after an order line is added, and raise low-stock alert
CREATE TRIGGER trg_reduce_stock
AFTER INSERT ON Order_Items
FOR EACH ROW
BEGIN
    DECLARE left_qty INT;
    DECLARE reorder INT;

    UPDATE Products
       SET stock_qty = stock_qty - NEW.quantity
     WHERE product_id = NEW.product_id;

    SELECT stock_qty, reorder_level INTO left_qty, reorder
    FROM Products WHERE product_id = NEW.product_id;

    IF left_qty <= reorder THEN
        INSERT INTO Stock_Alerts (product_id, stock_left)
        VALUES (NEW.product_id, left_qty);
    END IF;
END//

-- 3c. Mark an order as Paid when full payment has been received
CREATE TRIGGER trg_mark_paid
AFTER INSERT ON Payments
FOR EACH ROW
BEGIN
    DECLARE paid DECIMAL(12,2);
    DECLARE total DECIMAL(12,2);

    SELECT SUM(amount) INTO paid FROM Payments WHERE order_id = NEW.order_id;
    SELECT total_amount INTO total FROM Orders WHERE order_id = NEW.order_id;

    IF paid >= total THEN
        UPDATE Orders SET status = 'Paid' WHERE order_id = NEW.order_id;
    END IF;
END//
```

4. Stored Procedures (with Transactions)

```
-- 4a. Place an order with one product. All-or-nothing via transaction.
```

```
CREATE PROCEDURE place_order(
    IN  p_customer_id INT,
    IN  p_product_id  INT,
    IN  p_quantity    INT,
    OUT p_order_id    INT
)
BEGIN
    DECLARE v_price DECIMAL(10,2);

    -- If anything fails (e.g. insufficient stock), undo everything
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SET p_order_id = NULL;
        SELECT 'Order FAILED - rolled back' AS result;
    END;

    START TRANSACTION;

        SELECT price INTO v_price FROM Products WHERE product_id = p_product_id;

        INSERT INTO Orders (customer_id) VALUES (p_customer_id);
        SET p_order_id = LAST_INSERT_ID();

        INSERT INTO Order_Items (order_id, product_id, quantity, unit_price)
        VALUES (p_order_id, p_product_id, p_quantity, v_price);   -- triggers fire here

        UPDATE Orders
           SET total_amount = v_price * p_quantity
         WHERE order_id = p_order_id;

    COMMIT;
    SELECT CONCAT('Order ', p_order_id, ' placed successfully') AS result;
END//

-- 4b. Add another item to an existing pending order
CREATE PROCEDURE add_item_to_order(
    IN p_order_id   INT,
    IN p_product_id INT,
    IN p_quantity   INT
)
BEGIN
    DECLARE v_price DECIMAL(10,2);

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SELECT 'Could not add item - rolled back' AS result;
    END;

    START TRANSACTION;
        SELECT price INTO v_price FROM Products WHERE product_id = p_product_id;

        INSERT INTO Order_Items (order_id, product_id, quantity, unit_price)
        VALUES (p_order_id, p_product_id, p_quantity, v_price);

        UPDATE Orders
           SET total_amount = (SELECT SUM(quantity * unit_price)
                               FROM Order_Items WHERE order_id = p_order_id)
         WHERE order_id = p_order_id;
    COMMIT;
    SELECT 'Item added' AS result;
END//

-- 4c. Cancel an order and put stock back
CREATE PROCEDURE cancel_order(IN p_order_id INT)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SELECT 'Cancel failed' AS result;
    END;

    START TRANSACTION;
        UPDATE Products p
          JOIN Order_Items oi ON oi.product_id = p.product_id
           SET p.stock_qty = p.stock_qty + oi.quantity
         WHERE oi.order_id = p_order_id;
        UPDATE Orders SET status = 'Cancelled' WHERE order_id = p_order_id;
```

```
    COMMIT;
    SELECT 'Order cancelled and stock restored' AS result;
END//

-- 4d. Record a payment
CREATE PROCEDURE make_payment(
    IN p_order_id INT,
    IN p_amount   DECIMAL(12,2),
    IN p_method   VARCHAR(20)
)
BEGIN
    INSERT INTO Payments (order_id, amount, method)
    VALUES (p_order_id, p_amount, p_method);   -- trg_mark_paid fires
    SELECT 'Payment recorded' AS result;
END//

-- 4e. Restock a product from its supplier
CREATE PROCEDURE restock_product(IN p_product_id INT, IN p_qty INT)
BEGIN
    UPDATE Products SET stock_qty = stock_qty + p_qty
     WHERE product_id = p_product_id;
    SELECT 'Stock updated' AS result;
END//

DELIMITER ;
```

5. Testing the System

```
-- Successful order: Amit buys 2 Wireless Mice
CALL place_order(1, 1, 2, @oid);
SELECT @oid AS new_order_id;

-- Add a second item to the same order
CALL add_item_to_order(@oid, 4, 1);

-- Pay for it (2*599 + 399 = 1597)
CALL make_payment(@oid, 1597.00, 'UPI');

-- Failed order: only 8 Desk Lamps in stock -> trigger blocks, transaction rolls back
CALL place_order(2, 5, 20, @oid2);

-- Order that drops stock below reorder level -> creates a Stock_Alert
CALL place_order(3, 5, 4, @oid3);

-- Cancel that order and restore stock
CALL cancel_order(@oid3);
```

6. Join Queries

```
-- 6a. Full order details (INNER JOINs across 4 tables)
SELECT o.order_id, c.name AS customer, p.name AS product,
       oi.quantity, oi.unit_price,
       oi.quantity * oi.unit_price AS line_total, o.status
FROM Orders o
JOIN Customers   c  ON c.customer_id = o.customer_id
JOIN Order_Items oi ON oi.order_id   = o.order_id
JOIN Products    p  ON p.product_id  = oi.product_id
ORDER BY o.order_id;

-- 6b. Customers who have never ordered (LEFT JOIN)
SELECT c.customer_id, c.name
FROM Customers c
LEFT JOIN Orders o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL;

-- 6c. Total spent per customer (excluding cancelled orders)
SELECT c.name, COUNT(o.order_id) AS orders, COALESCE(SUM(o.total_amount),0) AS total_spent
FROM Customers c
LEFT JOIN Orders o ON o.customer_id = c.customer_id AND o.status <> 'Cancelled'
GROUP BY c.customer_id, c.name
ORDER BY total_spent DESC;

-- 6d. Products at or below reorder level, with supplier to contact
SELECT p.name AS product, p.stock_qty, p.reorder_level,
       s.name AS supplier, s.phone
```

```
FROM Products p
JOIN Suppliers s ON s.supplier_id = p.supplier_id
WHERE p.stock_qty <= p.reorder_level;

-- 6e. Orders with payment status (LEFT JOIN so unpaid orders still show)
SELECT o.order_id, c.name AS customer, o.total_amount,
       COALESCE(SUM(pay.amount),0) AS paid,
       o.total_amount - COALESCE(SUM(pay.amount),0) AS balance_due
FROM Orders o
JOIN Customers c ON c.customer_id = o.customer_id
LEFT JOIN Payments pay ON pay.order_id = o.order_id
GROUP BY o.order_id, c.name, o.total_amount;

-- 6f. Best-selling products
SELECT p.name, SUM(oi.quantity) AS units_sold
FROM Products p
JOIN Order_Items oi ON oi.product_id = p.product_id
JOIN Orders o       ON o.order_id = oi.order_id AND o.status <> 'Cancelled'
GROUP BY p.product_id, p.name
ORDER BY units_sold DESC;

-- 6g. Stock alerts raised by the trigger
SELECT a.alert_id, p.name, a.stock_left, a.alert_time
FROM Stock_Alerts a
JOIN Products p ON p.product_id = a.product_id;
```